

ASTRA — Production System Architecture

Version: 1.0 · Date: 2026-05-19 · Status: Reference

Table of Contents

- 1. [System Overview](#)
- 2. [Database Schema](#)
- 3. [API Contracts](#)
- 4. [Authentication System](#)
- 5. [Session Lifecycle](#)
- 6. [Polar BLE Integration](#)
- 7. [Saarthi AI Architecture](#)
- 8. [Realtime System](#)
- 9. [Analytics Engine](#)
- 10. [Storage Architecture](#)
- 11. [Production Infrastructure](#)

1. System Overview

ASTRA is a mental performance training platform for competitive shooting sport athletes. It combines pre/post-session psychology check-ins, biometric streaming from Polar H10 devices, AI-powered coaching (Saarthi), and longitudinal analytics into a single mobile-first product.

Core Components

Layer	Technology	Purpose
Mobile App	Flutter 3.x + BLoC	Athlete-facing interface
Routing	GoRouter 14.x	Declarative navigation, auth guards
State	flutter_bloc 8.x	Feature-scoped BLoC per feature
DI	GetIt 8.x	Lazy singletons + factory BLoCs
Local Storage	SharedPreferences	Offline-first session data
HTTP Client	Dio 5.x	Supabase REST, no SDK
Database	Supabase (PostgreSQL 15)	Primary data store
Time-series	TimescaleDB extension	HR/ECG/ACC telemetry hypertables
Realtime	Supabase Realtime (Phoenix)	Live session WebSocket events
Cache	Redis (Upstash)	AI context, hot dashboard data
Serverless	Supabase Edge Functions (Deno)	AI orchestration, analytics jobs

Layer	Technology	Purpose
Queue	pgmq	Async insight generation
Scheduler	pg_cron	Nightly analytics, data retention
AI	Claude claude-sonnet-4-6	Saarthi conversational coach
BLE	Polar SDK	H10 HR/ECG/ACC streaming

Design Constraints

- **Offline-first:** All local UI flows proceed without network. API calls are non-blocking fire-and-forget.
- **No vendor lock-in to Supabase SDK:** All API calls are raw Dio HTTP to the Supabase REST endpoint.
- **Case-sensitive REST paths:** `App_Sessions` $\neq$ `app_sessions`. All path strings must match the exact Supabase table names.
- **No routing/UI modification:** GoRouter configuration, Astra branding, Saarthi branding, theme system, and scoring logic are frozen.

2. Database Schema

2.1 Identity & Access

```
-- Primary user record (Supabase Auth integration)
CREATE TABLE "App_Users" (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  auth_uid    uuid UNIQUE REFERENCES auth.users(id) ON DELETE CASCADE,
  phone       text UNIQUE,
  email       text UNIQUE,
  role        text NOT NULL DEFAULT 'athlete' CHECK (role IN
('athlete','coach','admin')),
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE athletes (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid UNIQUE REFERENCES "App_Users"(id) ON DELETE
CASCADE,
  athlete_id  text UNIQUE NOT NULL,                -- human-readable:
ATH001
  full_name   text NOT NULL,
  date_of_birth date,
  gender      text CHECK (gender IN ('male','female','other')),
  sport       text NOT NULL DEFAULT 'shooting',
  discipline  text,                                -- 10m air rifle, 25m
pistol, etc.
  experience_years int,
  avatar_url  text,
  created_at  timestampz NOT NULL DEFAULT now()
```

```

);

CREATE TABLE athlete_details (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  athlete_id  uuid UNIQUE REFERENCES athletes(id) ON DELETE CASCADE,
  height_cm   numeric(5,1),
  weight_kg    numeric(5,1),
  dominant_hand text CHECK (dominant_hand IN ('left','right')),
  coach_id    uuid REFERENCES coaches(id),
  academy_id  uuid REFERENCES academies(id),
  bio         text,
  goals       text[],
  updated_at  timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE athlete_family (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  athlete_id  uuid REFERENCES athletes(id) ON DELETE CASCADE,
  name        text NOT NULL,
  relation    text NOT NULL,
  phone       text,
  is_emergency boolean DEFAULT false
);

CREATE TABLE coaches (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid UNIQUE REFERENCES "App_Users"(id),
  full_name   text NOT NULL,
  certifications text[],
  specialization text
);

CREATE TABLE academies (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL,
  city        text,
  state       text,
  country     text DEFAULT 'India'
);

```

2.2 Psychology & Check-in

```

CREATE TABLE psychology_questions (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  dimension   text NOT NULL,           -- focus, confidence,
anxiety, motivation, resilience
  question_text text NOT NULL,
  question_type text NOT NULL DEFAULT 'likert_5', -- likert_5, boolean,
open_text
  display_order int NOT NULL,
  is_active    boolean NOT NULL DEFAULT true
);

```

```

);

CREATE TABLE psychology_responses (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  athlete_id  uuid REFERENCES athletes(id) ON DELETE CASCADE,
  question_id uuid REFERENCES psychology_questions(id),
  session_id  uuid REFERENCES "App_Sessions"(id),
  response_value text NOT NULL,          -- numeric string for
likert, 'true'/'false', free text
  responded_at timestampz NOT NULL DEFAULT now()
);

CREATE TABLE psychology_scores (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  athlete_id  uuid REFERENCES athletes(id) ON DELETE CASCADE,
  session_id  uuid REFERENCES "App_Sessions"(id),
  focus_score  numeric(4,2),
  confidence_score numeric(4,2),
  anxiety_score  numeric(4,2),
  motivation_score numeric(4,2),
  resilience_score numeric(4,2),
  composite_score numeric(4,2),
  scored_at    timestampz NOT NULL DEFAULT now()
);

CREATE TABLE daily_checkins (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  athlete_id  uuid REFERENCES athletes(id) ON DELETE CASCADE,
  checkin_date date NOT NULL DEFAULT CURRENT_DATE,
  mood        int CHECK (mood BETWEEN 1 AND 10),
  energy      int CHECK (energy BETWEEN 1 AND 10),
  sleep_quality int CHECK (sleep_quality BETWEEN 1 AND 10),
  sleep_hours  numeric(3,1),
  stress_level int CHECK (stress_level BETWEEN 1 AND 10),
  soreness     int CHECK (soreness BETWEEN 1 AND 10),
  notes       text,
  created_at   timestampz NOT NULL DEFAULT now(),
  UNIQUE (athlete_id, checkin_date)
);

```

2.3 Session Management

```

CREATE TABLE "App_Sessions" (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  athlete_id  text NOT NULL,          -- athlete_id string,
e.g. ATH001
  session_type text NOT NULL CHECK (session_type IN
('training', 'competition', 'practice')),
  range_type  text CHECK (range_type IN ('paper', 'electronic')),
  planned_shots int,
  status      text NOT NULL DEFAULT 'pending'
);

```

```

        CHECK (status IN
('pending','active','paused','completed','aborted')),
    start_time      timestamptz,
    end_time        timestamptz,
    duration_mins   int GENERATED ALWAYS AS (
        EXTRACT(epoch FROM (end_time - start_time))/60
    ) STORED,
    notes           text,
    created_at      timestamptz NOT NULL DEFAULT now()
);

-- Series-level score data
CREATE TABLE session_series (
    id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id      uuid REFERENCES "App_Sessions"(id) ON DELETE CASCADE,
    series_number   int NOT NULL,
    shots           text[] NOT NULL,                -- individual shot
values as strings
    total           numeric(6,2) NOT NULL,
    avg_per_shot    numeric(5,2),
    created_at      timestamptz NOT NULL DEFAULT now(),
    UNIQUE (session_id, series_number)
);

-- Aggregate performance per session
CREATE TABLE performance_summary (
    id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    athlete_id      text NOT NULL,
    session_id      uuid REFERENCES "App_Sessions"(id),
    total_score     numeric(8,2),
    max_possible    numeric(8,2),
    avg_per_shot    numeric(5,2),
    score_variance  numeric(8,4),
    consistency_pct numeric(5,2),
    performance_rating int CHECK (performance_rating BETWEEN 1 AND 10),
    focus_level     text CHECK (focus_level IN
('low','moderate','high')),
    created_at      timestamptz NOT NULL DEFAULT now()
);

-- Post-session reflection
CREATE TABLE session_reflections (
    id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id      uuid REFERENCES "App_Sessions"(id) ON DELETE
CASCADE,
    athlete_id      uuid REFERENCES athletes(id),
    what_went_well  text,
    what_to_improve text,
    key_takeaway    text,
    emotion_rating  int CHECK (emotion_rating BETWEEN 1 AND 5),
    created_at      timestamptz NOT NULL DEFAULT now()
);

```

2.4 Biometric Telemetry (TimescaleDB)

```

-- Heart rate samples (1 Hz nominal)
CREATE TABLE hr_stream (
  time          timestamptz NOT NULL,
  session_id    uuid NOT NULL,
  athlete_id    text NOT NULL,
  bpm           int NOT NULL,
  rr_interval    int,                      -- ms between beats
  source        text DEFAULT 'polar_h10'
);
SELECT create_hypertable('hr_stream','time');
CREATE INDEX ON hr_stream (session_id, time DESC);

-- ECG raw samples (130 Hz from Polar H10)
CREATE TABLE ecg_stream (
  time          timestamptz NOT NULL,
  session_id    uuid NOT NULL,
  athlete_id    text NOT NULL,
  microvolts    int NOT NULL,
  sample_rate   int DEFAULT 130
);
SELECT create_hypertable('ecg_stream','time', chunk_time_interval =>
INTERVAL '1 hour');
CREATE INDEX ON ecg_stream (session_id, time DESC);

-- Accelerometer (200 Hz from Polar H10)
CREATE TABLE acc_stream (
  time          timestamptz NOT NULL,
  session_id    uuid NOT NULL,
  athlete_id    text NOT NULL,
  x             int NOT NULL,             -- mG
  y             int NOT NULL,
  z             int NOT NULL,
  magnitude     numeric(8,3) GENERATED ALWAYS AS (
    sqrt(x*x + y*y + z*z)::numeric(8,3)
  ) STORED
);
SELECT create_hypertable('acc_stream','time', chunk_time_interval =>
INTERVAL '1 hour');
CREATE INDEX ON acc_stream (session_id, time DESC);

-- Retention policies (TimescaleDB)
SELECT add_retention_policy('hr_stream', INTERVAL '2 years');
SELECT add_retention_policy('ecg_stream', INTERVAL '6 months');
SELECT add_retention_policy('acc_stream', INTERVAL '6 months');

-- Continuous aggregates for dashboard queries (avoid full-table scan)
CREATE MATERIALIZED VIEW hr_minutely
  WITH (timescaledb.continuous) AS
  SELECT time_bucket('1 minute', time) AS bucket,
         session_id,

```

```

        avg(bpm)::int AS avg_bpm,
        min(bpm)      AS min_bpm,
        max(bpm)      AS max_bpm
FROM hr_stream
GROUP BY bucket, session_id;

```

2.5 AI & Coach

```

CREATE TABLE ai_insights (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  athlete_id  text NOT NULL,
  session_id  uuid REFERENCES "App_Sessions"(id),
  insight_type text NOT NULL CHECK (insight_type IN
('performance','mental','recovery','pattern','recommendation')),
  title       text NOT NULL,
  body        text NOT NULL,
  priority    int DEFAULT 5 CHECK (priority BETWEEN 1 AND 10),
  is_read     boolean DEFAULT false,
  generated_at timestampz NOT NULL DEFAULT now()
);

CREATE TABLE athlete_chat (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  athlete_id  uuid REFERENCES athletes(id) ON DELETE CASCADE,
  session_id  uuid REFERENCES "App_Sessions"(id),
  role        text NOT NULL CHECK (role IN
('user','assistant','system')),
  content     text NOT NULL,
  token_count int,
  created_at  timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ON athlete_chat (athlete_id, created_at DESC);

CREATE TABLE insight_library (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  category    text NOT NULL,                -- breathing, focus,
visualization, routine
  title       text NOT NULL,
  body        text NOT NULL,
  audio_url   text,
  duration_secs int,
  tags        text[],
  display_order int DEFAULT 0
);

CREATE TABLE audio_library (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title       text NOT NULL,
  description  text,
  url         text NOT NULL,

```

```

    duration_secs    int,
    category         text,
    is_active        boolean DEFAULT true
);

CREATE TABLE coach_feedback (
    id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    athlete_id       text NOT NULL,
    coach_id         uuid REFERENCES coaches(id),
    session_id       uuid REFERENCES "App_Sessions"(id),
    feedback_type    text CHECK (feedback_type IN
('technical','mental','tactical','general')),
    content          text NOT NULL,
    rating           int CHECK (rating BETWEEN 1 AND 5),
    created_at       timestampz NOT NULL DEFAULT now()
);

```

2.6 Polar Device Registry

```

CREATE TABLE polar_devices (
    id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    athlete_id       uuid REFERENCES athletes(id) ON DELETE CASCADE,
    device_id        text NOT NULL,                -- Polar device ID
    string
    device_name      text,
    model            text DEFAULT 'H10',
    firmware_version text,
    last_connected   timestampz,
    is_primary       boolean DEFAULT true,
    created_at       timestampz NOT NULL DEFAULT now(),
    UNIQUE (athlete_id, device_id)
);

```

2.7 RLS Policies (Row-Level Security)

```

-- Athletes can only read/write their own rows
ALTER TABLE "App_Sessions" ENABLE ROW LEVEL SECURITY;
CREATE POLICY athlete_own_sessions ON "App_Sessions"
    USING (athlete_id = current_setting('app.athlete_id', true));

ALTER TABLE hr_stream ENABLE ROW LEVEL SECURITY;
CREATE POLICY athlete_own_hr ON hr_stream
    USING (athlete_id = current_setting('app.athlete_id', true));

-- Coaches can read athletes they manage
CREATE POLICY coach_read_sessions ON "App_Sessions"
    FOR SELECT
    USING (
        athlete_id IN (

```



```
SELECT a.athlete_id FROM athletes a
JOIN athlete_details ad ON ad.athlete_id = a.id
WHERE ad.coach_id = (
    SELECT id FROM coaches WHERE user_id = auth.uid()
)
);
```

3. API Contracts

3.1 Path Convention

```
baseUrl = https://pjjkfnofislpdckjixtg.supabase.co/rest/v1/
```

Headers (all requests):

apikey: <SUPABASE_ANON_KEY>

Authorization: Bearer <SUPABASE_ANON_KEY> (or user JWT after auth)

Content-Type: application/json

PATH RULE: Never prefix with '/'. baseUrl already ends with '/'.
'App_Sessions' → https://.../rest/v1/App_Sessions ✓
'/App_Sessions' → https://.../App_Sessions x

3.2 Session APIs

Create session

```
POST /rest/v1/App_Sessions
```

Headers: Prefer: return=representation

Body:

```
{
  "athlete_id": "ATH001",
  "session_type": "training",
  "range_type": "paper",
  "planned_shots": 60,
  "start_time": "2026-05-19T08:30:00Z",
  "status": "active"
}
```

Response 201:

```
[{
  "id": "<uuid>",
  "athlete_id": "ATH001",
  "session_type": "training",
  "status": "active",
  "start_time": "2026-05-19T08:30:00Z",
  "created_at": "2026-05-19T08:30:00Z"
}]
```

Complete session

```
PATCH /rest/v1/App_Sessions?id=eq.<session_id>
Body:
{
  "status": "completed",
  "end_time": "2026-05-19T10:15:00Z"
}
Response 204
```

Get athlete sessions

```
GET /rest/v1/App_Sessions?
athlete_id=eq.ATH001&order=created_at.desc&limit=20
Response 200: [{...}, ...]
```

3.3 Athlete APIs

Get full athlete profile

```
GET /rest/v1/athletes?athlete_id=eq.ATH001
  &select=*,athlete_details(*),athlete_family(*)
Response 200:
[
  {
    "id": "<uuid>",
    "athlete_id": "ATH001",
    "full_name": "...",
    "athlete_details": { "height_cm": 175, "coach_id": "...", ... },
    "athlete_family": [ { "name": "...", "relation": "father", ... } ]
  }
]
```

3.4 Check-in APIs

Save daily check-in

```
POST /rest/v1/daily_checkins
Headers: Prefer: return=representation
Body:
{
  "athlete_id": "<uuid>",
  "checkin_date": "2026-05-19",
  "mood": 7,
  "energy": 6,
  "sleep_quality": 8,
  "sleep_hours": 7.5,
  "stress_level": 4,
}
```

```
"soreness":    3
}
```

Get today's check-in

```
GET /rest/v1/daily_checkins?athlete_id=eq.<uuid>&checkin_date=eq.2026-05-19
```

3.5 Performance & Analytics APIs

Save performance summary

```
POST /rest/v1/performance_summary
Headers: Prefer: return=representation
Body:
{
  "athlete_id":    "ATH001",
  "session_id":    "<uuid>",
  "total_score":   524.3,
  "max_possible":  600.0,
  "avg_per_shot":  8.74,
  "score_variance": 0.34,
  "consistency_pct": 91.2,
  "performance_rating": 8,
  "focus_level":  "high"
}
```

Get performance history

```
GET /rest/v1/performance_summary?
athlete_id=eq.ATH001&order=created_at.desc&limit=30
```

3.6 Telemetry APIs (Bulk Insert)

Bulk insert HR samples

```
POST /rest/v1/hr_stream
Headers: Prefer: return=minimal
Body:
[
  { "time": "2026-05-19T08:30:01Z", "session_id": "<uuid>", "athlete_id": "ATH001", "bpm": 68, "rr_interval": 882 },
  { "time": "2026-05-19T08:30:02Z", "session_id": "<uuid>", "athlete_id": "ATH001", "bpm": 69, "rr_interval": 870 }
]
```

Note: Buffer at least 10 seconds locally before bulk flushing to reduce write amplification.

3.7 AI Insights APIs

Get athlete insights

```
GET /rest/v1/ai_insights?
athlete_id=eq.ATH001&is_read=eq.false&order=priority.desc&limit=10
```

Mark insight read

```
PATCH /rest/v1/ai_insights?id=eq.<uuid>
Body: { "is_read": true }
```

3.8 Saarthi Chat API (Edge Function)

```
POST /functions/v1/saarthi-chat
Headers: Authorization: Bearer <user_jwt>
Body:
{
  "athlete_id": "<uuid>",
  "session_id": "<uuid>",          // optional – null outside sessions
  "message":    "I feel nervous before competitions",
  "context_mode": "pre_session"    // pre_session | post_session |
general
}
Response:
{
  "reply":      "That nervousness is your body preparing ...",
  "suggestions": ["Try box breathing", "Review your pre-shot routine"],
  "mood_flag":   null              // "high_anxiety" | "low_motivation" |
null
}
```

3.9 RPC (Stored Procedures)

Dashboard aggregates

```
POST /rest/v1/rpc/get_dashboard_data
Body: { "p_athlete_id": "ATH001" }
Response:
{
  "avg_score_7d":      513.4,
  "consistency_7d":    88.3,
```

```

    "sessions_this_week": 4,
    "readiness_score": 76,
    "trend": "improving"
  }

```

Note: Existence of this RPC function is unconfirmed. Caller must catch errors gracefully.

3.10 Disabled Endpoints (Tables Not Yet Created)

The following tables and endpoints are commented out in the codebase pending creation in Supabase:

- `session_pre_log` — pre-session psychology log linked to session
- `session_post_log` — post-session debrief log

Do not call these until the tables are confirmed in Supabase.

4. Authentication System

4.1 Token Architecture

Auth Flow

1. Phone OTP (Supabase Auth)
 - POST `/auth/v1/otp` { phone: "+91..." }
 - POST `/auth/v1/verify` { phone, token, type: "sms" }
2. Response:
 - `access_token` — JWT RS256, 15-min TTL
 - `refresh_token` — opaque, 30-day TTL, rotating
3. Token storage: SharedPreferences (local, encrypted)
4. Refresh: POST `/auth/v1/token?grant_type=refresh_token`
→ new `access_token` + new `refresh_token`

4.2 JWT Claims

```

{
  "sub": "<auth.users.id>",
  "role": "authenticated",
  "app_metadata": {
    "athlete_id": "ATH001",
    "role": "athlete"
  },
  "exp": 1716211200
}

```

4.3 RLS Integration

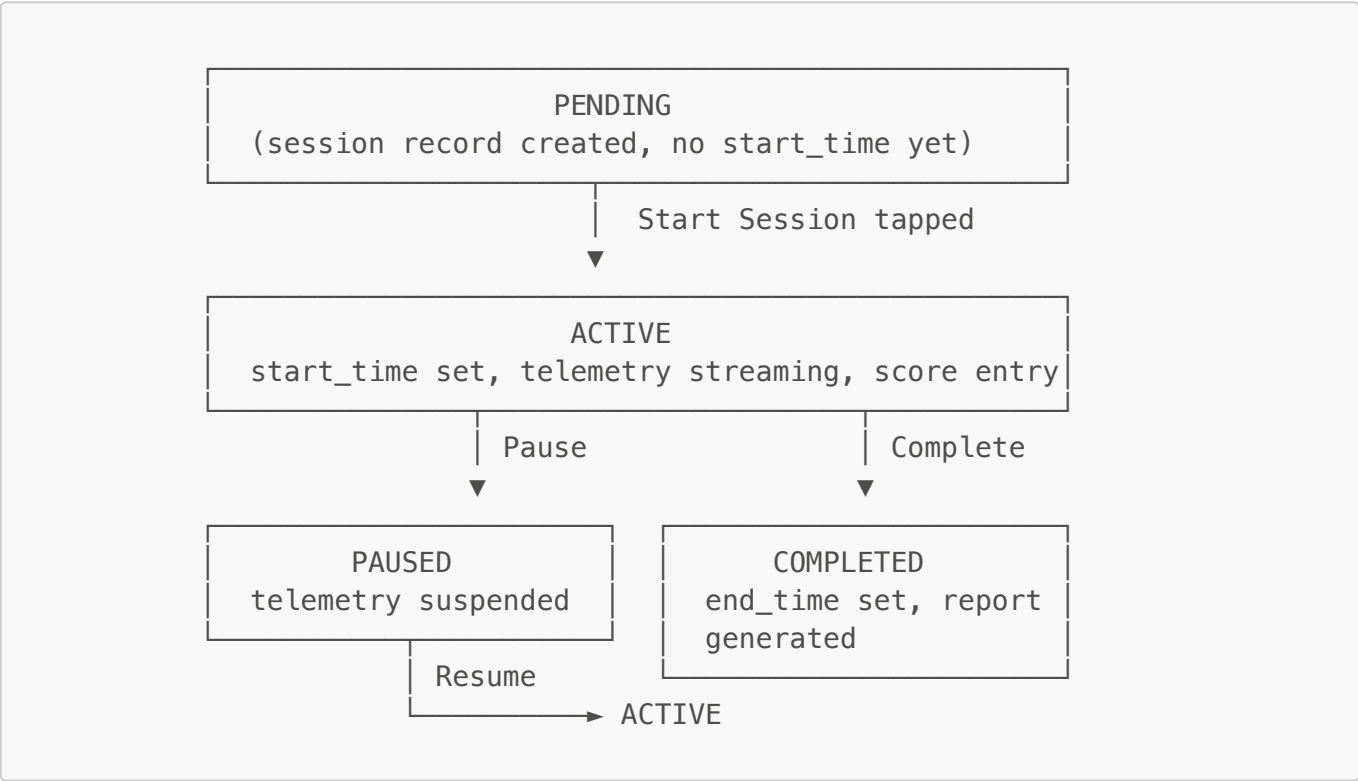
After login, all Supabase REST requests carry:

```
Authorization: Bearer <access_token>
```

RLS policies evaluate `auth.uid()` and `current_setting('app.athlete_id')` to scope data access per athlete row.

5. Session Lifecycle

5.1 State Machine



5.2 Session Phases in App

Phase	Screens	Local Data	API Calls
Setup	SessionSetup	sessionSetup → SharedPrefs	—
Pre-ritual	PreSessionRitual	—	—
Pre-checkin	CheckIn	checkin → SharedPrefs	POST daily_checkins
Psychology	Psychology questions	psych responses → SharedPrefs	POST psychology_responses

Phase	Screens	Local Data	API Calls
Session	ScoreEntry + LiveTraining	series scores → SharedPrefs	POST App_Sessions (start)
Post	Reflection	reflection → SharedPrefs	PATCH App_Sessions (complete)
Summary	SessionSummary + Report	—	POST performance_summary

5.3 Non-Blocking API Pattern

All API calls follow this pattern to ensure local flow is never gated on network:

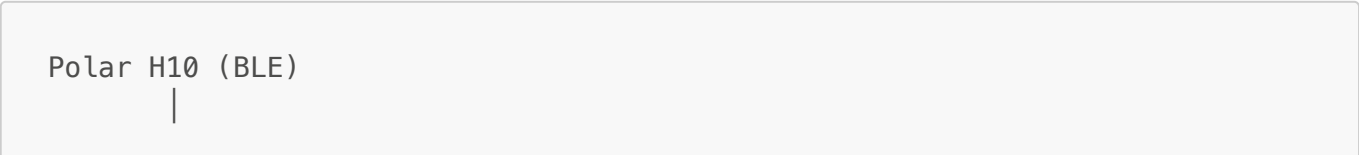
```
void _syncToApi() {
  Future() async {          // fire-and-forget: does not block UI
    try {
      await ApiService.instance.someCall(...);
    } catch (e) {
      debugPrint('[SYNC ERROR] $e');
      // silent failure – local data is the source of truth
    }
  });
}
```

6. Polar BLE Integration

6.1 Device Pairing Flow

```
1. Scan for nearby Polar devices (PolarBleApi.searchForDevice)
2. User selects device from list
3. Connect: PolarBleApi.connectToDevice(deviceId)
4. Subscribe to streams:
  - HR stream: PolarBleApi.startHrStreaming(deviceId)
  - ECG stream: PolarBleApi.startEcgStreaming(deviceId,
PolarSensorSetting)
  - ACC stream: PolarBleApi.startAccStreaming(deviceId,
PolarSensorSetting)
5. Store deviceId in SharedPreferences
6. Register in polar_devices table
```

6.2 Streaming Architecture





6.3 Buffer Flush Strategy

```
class TelemetryBuffer {
  static const _hrFlushInterval = Duration(seconds: 10);
  static const _ecgFlushInterval = Duration(seconds: 5);
  static const _maxBatchSize = 500; // rows per POST

  // Flush on: interval elapsed OR buffer reaches maxBatchSize OR session
  ends
}
```

6.4 Sensor Settings (Polar H10)

Stream	Sample Rate	Resolution	Range
HR	1 Hz (interrupt-driven)	8-bit BPM	30–240 bpm
ECG	130 Hz	14-bit	±1 mV
ACC	200 Hz	16-bit	±8 G (configurable)

6.5 Reconnection Logic

```
On disconnect event:
  1. Log disconnect time
  2. Wait 2 seconds (debounce)
  3. Retry connect (up to 5 attempts, exponential backoff: 2s, 4s, 8s,
  16s, 30s)
  4. On reconnect: resume streaming, continue buffering
```


5. On permanent failure: store remaining buffer locally, flush on next opportunity

7. Saarthi AI Architecture

7.1 System Overview

Saarthi is an AI performance coach powered by Claude claude-sonnet-4-6. It maintains rolling conversational context per athlete across sessions.

7.2 System Prompt Template

You are Saarthi, an elite mental performance coach for competitive shooting athletes.

You speak with calm authority, warmth, and precision.

You draw from sport psychology, mindfulness, and performance science.

Never diagnose. Never replace a human coach. Always empower the athlete.

Athlete context:

- Name: {athlete.full_name}
- Discipline: {athlete.discipline}
- Experience: {athlete.experience_years} years
- Recent avg score: {performance.avg_score_7d}
- Consistency: {performance.consistency_7d}%
- Today's mood: {checkin.mood}/10, energy: {checkin.energy}/10, sleep: {checkin.sleep_hours}h

Session context (if active):

- Session type: {session.session_type}
- Planned shots: {session.planned_shots}
- Current score trend: {score.trend}

Recent insights:

{ai_insights.last_3}

Conversation history (rolling, last 20 turns):

{chat_history}

7.3 Context Memory Architecture

Saarthi Context Pipeline (Edge Function)

1. Fetch rolling context (Redis, TTL 24h):

key = saarthi:context:{athlete_id}

value = last 20 chat turns + athlete snapshot

2. Cache miss → rebuild from Supabase:

```
SELECT * FROM athlete_chat
WHERE athlete_id = $1
ORDER BY created_at DESC LIMIT 20
```

3. Inject athlete snapshot + check-in + session context
4. Call Claude API (claude-sonnet-4-6)
max_tokens: 1024, temperature: 0.7
5. Persist exchange: INSERT INTO athlete_chat (user + AI)
6. Update Redis cache
7. Flag mood signals → queue insight generation if needed

7.4 Insight Generation Pipeline

Trigger: session completed OR scheduled nightly job

1. Edge Function: generate-insights
2. Gather last 30 days: sessions, scores, check-ins, psych scores, HR data
3. Claude prompt: "Identify 3 actionable insights for {athlete}. JSON output."
4. Parse response → INSERT INTO ai_insights (3 rows)
5. Mark old unread insights as superseded
6. Push notification to athlete (if subscribed)

8. Realtime System

8.1 WebSocket Architecture

Flutter App

Supabase Realtime WebSocket
wss://pjkkjnofislpdckjixtg.supabase.co/realtime/v1



RealtimeChannel: session:{session_id}

Events:

postgres_changes → App_Sessions (UPDATE status)
postgres_changes → ai_insights (INSERT for live coaching cues)
broadcast → telemetry:summary (coach live view)

8.2 Channel Subscription (Dart)

```
// Subscribe to live session updates
final channel = supabase.channel('session:$sessionId')
  ..on(
    RealtimeListenTypes.postgresChanges,
    ChannelFilter(event: 'UPDATE', schema: 'public', table:
'App_Sessions', filter: 'id=eq.$sessionId'),
    (payload, [ref]) {
      final updated = payload['new'] as Map<String, dynamic>;
      sessionBloc.add(SessionRemoteUpdated(updated));
    },
  )
  ..subscribe();
```

Note: This project uses raw Dio for REST. Realtime would require the [supabase_flutter](#) SDK or manual Phoenix channel WebSocket client. Evaluate before implementing.

9. Analytics Engine

9.1 Score Analytics

Performance Rating (1–10)

```
total      = sum of all series totals
max_total  = series_count × shots_per_series × 10.9  (10.9 = max shot
value)
raw_rating = (total / max_total) × 10
rating     = clamp(round(raw_rating), 1, 10)
```

Focus Level

```
rating >= 7 → "high"
rating >= 5 → "moderate"
rating < 5  → "low"
```

Score Variance

```
avg  = total_score / shot_count
var  = mean((shot_i - avg)^2 for all shots)
```

Consistency %

```
consistency = (1 - (std_dev / avg)) × 100
```

Avg Per Shot

```
avg_per_shot = total_score / total_shots
```

9.2 Readiness Score (0–100)

```
readiness = weighted sum:  
  sleep_quality × 0.30  
  energy        × 0.25  
  mood          × 0.20  
  (10 - stress) × 0.15    // inverted: low stress = high readiness  
  (10 - soreness) × 0.10  // inverted
```

All inputs on 1–10 scale → normalize each to 0–10 → apply weights → multiply by 10

9.3 Recovery Score

```
recovery = (sleep_hours / 8) × 40           // sleep contribution  
(max 40)  
          + ((10 - soreness) / 10) × 30    // soreness  
contribution (max 30)  
          + ((10 - stress) / 10) × 30      // stress  
contribution (max 30)  
clamp(recovery, 0, 100)
```

9.4 Mental State Index

```
mental_state = mean(focus_score, confidence_score, motivation_score)  
              × (1 - anxiety_score / 10 × 0.3)    // anxiety penalty
```

Output: 0–10 scale

9.5 Psychology Dimension Scores

```
For each dimension D (focus, confidence, anxiety, motivation, resilience):  
  responses = all likert_5 responses for questions in dimension D  
  raw_score = mean(response_value for each response) // 1–5 scale  
  normalized = (raw_score - 1) / 4 × 10             // 0–10 scale
```

9.6 HR Analytics (per session)

```
avg_hr      = mean(bpm) for session duration
max_hr      = max(bpm)
min_hr      = min(bpm)
hr_variability = std_dev(rr_interval) // HRV proxy
pre_shot_hr = avg(bpm in 5s window before each shot)
hr_recovery = time from peak to return within 10% of resting
```

9.7 Trend Analysis (7-day)

```
For metric M over last 7 sessions:
slope = linear_regression_slope([M1, M2, ... M7])
trend = "improving" if slope > 0.05 × baseline
       "stable"    if |slope| ≤ 0.05 × baseline
       "declining" if slope < -0.05 × baseline
```

10. Storage Architecture

10.1 Local Storage (SharedPreferences)

Keys in use:

auth_token	→ JWT access token
refresh_token	→ opaque refresh token
onboarding_step	→ int (0–N)
onboarding_complete	→ bool
athlete_profile	→ JSON string
session_setup	→ JSON string (rangeType, sessionType, plannedShots)
score_summary	→ JSON string (list of series objects)
current_checkin	→ JSON string
journal_entries	→ JSON string (list)
baseline_scores	→ JSON string
polar_device_id	→ string

10.2 Supabase Storage Buckets

Bucket: athlete-avatars
Path: {athlete_id}/avatar.jpg
Access: Private (signed URL, 1h TTL)
Max size: 5 MB

Bucket: audio-content
Path: {category}/{filename}.mp3
Access: Public
CDN: Supabase CDN (edge-cached)

```
Bucket: session-exports
  Path: {athlete_id}/{session_id}/report.pdf
  Access: Private (signed URL)
  Retention: 90 days
```

10.3 Redis Cache (Upstash)

```
saarthi:context:{athlete_id}    TTL: 24h    Size: ~4KB
dashboard:data:{athlete_id}     TTL: 5m     Size: ~1KB
athlete:profile:{athlete_id}    TTL: 1h     Size: ~2KB
insight:pending:{athlete_id}    TTL: 1h     Size: ~500B
```

10.4 TimescaleDB Retention

```
hr_stream:    2 years (high value for longitudinal HRV analysis)
ecg_stream:   6 months (large: 130 Hz × session duration)
acc_stream:   6 months (large: 200 Hz × session duration)
```

Continuous aggregates retained indefinitely:

```
hr_minutely   → dashboard HR charts
acc_daily     → movement load trends
```

11. Production Infrastructure

11.1 Phase 1 — MVP (Current)

```
Supabase Hosted (Pro plan)
├─ PostgreSQL 15 + TimescaleDB
├─ Supabase Auth (phone OTP)
├─ Supabase REST API (primary interface)
├─ Supabase Storage (avatars, audio)
└─ Supabase Edge Functions (Saarthi, insights)
```

Redis: Upstash (serverless, pay-per-request)

Mobile: Flutter (iOS + Android)
BLE: Polar SDK (H10 only)

11.2 Phase 2 — Growth (1k–10k athletes)

```
Add:
├─ Supabase Realtime → live coach dashboard
```

- └─ pgmq → async insight generation queue
- └─ pg_cron → nightly analytics jobs
- └─ Supabase CDN → audio library streaming
- └─ Push notifications → APNs + FCM via Supabase

11.3 Phase 3 — Scale (10k+ athletes)

Add:

- └─ Read replicas → analytics queries (avoid write path)
- └─ TimescaleDB compression → reduce hot storage 80%
- └─ Redis cluster → distributed caching
- └─ API Gateway → rate limiting, DDoS protection
- └─ Multi-region Supabase → latency SLA for international athletes

11.4 Monitoring

Application:

Flutter: Sentry (crash reporting, performance tracing)

Backend:

Supabase Dashboard: query performance, RLS audit

pg_stat_statements: slow query identification

TimescaleDB telemetry: chunk health, compression ratio

Alerts:

API error rate > 1% → PagerDuty

p99 latency > 2s → PagerDuty

ECG buffer lag > 10s → Slack

11.5 Security Checklist

- ✓ RLS enabled on all athlete-data tables
- ✓ Anon key: read-only public tables only
- ✓ Service role key: never shipped to mobile client
- ✓ JWT RS256: 15-min TTL, no symmetric secrets in app
- ✓ Refresh tokens: rotating, single-use
- ✓ BLE: device ID stored locally, not transmitted to third parties
- ✓ AI chat: no PII in Claude API calls (athlete_id only, no names)
- ✓ Storage: private buckets, signed URLs only
- ✓ Supabase SSL enforced (TLS 1.2+)

Appendix A — Flutter File Map



Item	Blocker	Priority
Polar BLE streaming + TelemetryBuffer	Polar SDK integration	Medium
Saarathi Edge Function deployment	Edge Function code	Medium
Realtime channel subscription	supabase_flutter SDK or Phoenix WS	Low
pg_cron nightly analytics job	Phase 2	Low

Document generated: 2026-05-19 · ASTRA v1.0 · Do not commit credentials to version control.